

AI Intelligence Project Handoff

Last updated: 2026-09-17

This is the primary handoff for continuing the AI Intelligence project in another assistant/chat. It reflects the deployed Serbia POC on remote main, including evidence implementation baseline 9e6cef1, investigation memory, scenario playback, bilingual UI behavior, mobile run recovery, multiplexed General/Moshe/Talia routing, the V2.1 Evidence Layer, Moshe target workflows, Talia enemy assessments, and additive result-layer presentation.

One-Line Summary

The active project is the Serbia/North Kosovo intelligence-analysis POC: a bilingual analyst UI backed by a multiplexed Hermes gateway and isolated General, Moshe, and Talia profiles over the 14,800-record V2.1 synthetic dataset. It supports raw data exploitation, semantic retrieval, fused evidence, target candidates, enemy assessments, MIL-STD-oriented evidence symbols, assessment graphics, investigation memory, scenario playback, and additive map/timeline/table layers.

Repository And Current State

Repository:

- <https://github.com/zvipev10/AI-Intelligence>

Current branch:

- Evidence implementation baseline 9e6cef1 is merged into remote main; later documentation-only commits may follow it.
- The local working checkout may remain on codex/evidence-semantic-fusion; fetch and inspect before switching.
- The older branch references in this handoff are historical only; do not assume they reflect the current checked-out branch.
- Always fetch before continuing and verify the current head with `git log -1 --oneline`.

Important local workspace rule:

- Workspace paths vary by session; use the current repository root rather than a historical absolute path.
- The workspace may contain unrelated local files; do not stage or deploy them unless the user explicitly asks.

Current local working tree expectation:

- main should be aligned with origin/main; do not assume the current working branch or tree is clean.
- Do not continue from stale local files if `git fetch origin` shows the active remote branch ahead.
- Identify and preserve unrelated dirty files before staging, switching branches, or deploying.
- As of 2026-09-17, the local workspace contains unrelated, preserved mobile-recovery/deployment changes in `.ai/work/capabilities/mobile-agent-run-recovery/checkpoint-001.md`, `llm_investigation_orchestrator_serbia_poc/server.py`, and `llm_investigation_orchestrator_serbia_poc/deployment/SHA256SUMS-v184.txt`.
- An isolated English WIP sibling workspace exists outside the restored repo; do not stage or deploy it accidentally from the main repo.

Latest Update: Semantic Evidence Fusion And Talia Consumption

Date: 2026-09-17

The evidence preparation and fusion path was upgraded and deployed end to end.

Implemented behavior:

- prepare_evidence preserves structured object classes and resolves missing classes through the existing semantic concept vocabulary used by semantic retrieval.
- prepare_fused_evidence consumes those normalized records and retains the existing source-grouping and binary persistence rules.
- The extra ambiguity persistence block was removed as requested; no new provisional evidence status was added.
- Deployment catalog fusion now uses an eight-hour bounded rolling window rather than fixed wall-clock buckets.
- The Evidence Layer remains precomputed for demo responsiveness, but its fused objects are now a read seed of the MCP evidence repository. SQLite remains the writable overlay for later persisted evidence.
- get_evidence, search_evidence, assessment validation, provenance tracing, the UI Evidence Layer, and Talia now resolve the same deterministic fused IDs.
- Talia's profile now searches existing fused evidence first, creates missing evidence through the existing tools, and keeps different phenomena in separate evidence objects before assessing their operational relationship.
- General, Moshe, and Talia MCP environments are aligned to dataset V2.1 and the shared Hebrew evidence catalog.

Production deployment:

- Remote main: 9e6cef1
- UI catalog schema: evidence-catalog-v2
- Catalog counts: 14,800 source records, 4,500 cataloged observations/structured reports, 783 fused objects, 5,283 total rows.
- serbia-poc-ui.service: active.
- hermes-gateway.service: active.
- Gateway API key was preserved during deployment; no key rotation was performed.
- A backup was created before synchronizing multiplex profile entries:
/home/ubuntu/.hermes/config.yaml.before-evidence-profile-sync-1789668252.

Verified Ibar Bridge evidence:

- Armored vehicles: EVD-FUSED-CE661784179B3B88BCB9
- Helicopter activity: EVD-FUSED-FDF2C6DAD2CC106E7BD2
- Engineering activity: EVD-FUSED-C1E6F848E654359BD98C

The helicopter object includes the additional compatible UAV record REC-V2-007693 within the rolling window. This is retained transparently in provenance.

Production verification through Talia's deployed environment confirmed:

- active dataset v2.1;
- all three fused IDs are returned by search_evidence for ENT-KSF at LOC-V2-010;

- semantic preparation resolves REC-V2-014170 as משריין רכב, REC-V2-008463 as מסוק, and REC-V2-012738 as הנדסיות עבודות;
- the multiplex gateway remains active after configuration synchronization.

Authoritative capability artifacts:

- .ai/work/capabilities/evidence-semantic-fusion/status.md
- .ai/work/capabilities/evidence-semantic-fusion/checkpoint-001.md
- .ai/work/capabilities/evidence-semantic-fusion/handoff-summary.md

Recent Capabilities Since The Previous Project Handoff

The August handoff predates several deployed capabilities. Current main also includes:

- a precomputed, demand-opened Evidence Layer with bounded map rendering;
- raw-layer neutral presentation instead of treating every raw record as a military symbol;
- UAV raw-record video simulation in the entity viewer;
- Talia's durable enemy-assessment tools and assessment graphics;
- corrected movement/route-axis visualization constraints;
- result controls for catalog-backed answers and supporting evidence layers;
- null-safe result rendering;
- mobile recovery of agent runs after the browser loses focus;
- Moshe consolidation into the multiplexed main Hermes gateway and removal of General Persistent.

Sections below this point are historical and may describe superseded runtime states.

Historical Update: English Localization Hardening In Isolated WIP

Date: 2026-08-06

The isolated English WIP advanced from partial shell localization toward proper backend-served English behavior.

What was clarified and implemented in the isolated English WIP:

- a client-side translation patch for English leftovers was explicitly rejected;
- the correct direction is backend/data localization, not UI-only substitution;
- the English workstream now targets server-side localization of English responses, including:
 - source_type - source_types - recorded-answer and investigation-step text fields such as answer, result, action, decision, bridge_summary, and related step metadata;
 - the temporary frontend translation workaround in app.js was removed again after the backend path was introduced;
 - the English WIP server now localizes JSON payloads for lang=en before returning them to the client;
 - the English playback/recorded-run path was redirected to localize from the canonical recorded runs on the backend rather than relying on recorded_runs_en as an authoritative source.

Current status of that work:

- the backend-localization direction is the intended architecture for English mode;
- the isolated WIP still needs final validation with live English responses after deployment;
- the VM deployment of this latest English WIP was attempted but blocked in the current session because the VM SSH private key or real local .hermes-api.json was not present in the workspace.

Operational blocker encountered on 2026-08-06:

- the deploy script and VM target were confirmed:
- host: 151.145.93.180 - root: /opt/serbia-poc-ui - service: serbia-poc-ui.service
- however, only example Hermes config files were available locally;
- no deployable SSH key was available in the project, the broader AI Intelligence workspace, or C:\Users\user\.ssh;
- therefore this exact English WIP state was not deployed from this session.

Recommended next continuation step:

- restore access to the real VM SSH key path or the real local .hermes-api.json;
- deploy the isolated English WIP;
- validate the following in English mode against the deployed VM:
- source types in all visible surfaces; - “What came back” investigation-step content; - recorded runs; - workstream result tables; - language switching regression against Hebrew mode.

Previous Update: Hebrew Restore Deployed, English Work Isolated

Date: 2026-08-06

The public VM was restored to the last stable Hebrew build after a partial English localization branch was found to be incomplete for deployment.

- The public UI was redeployed from the committed Hebrew state on main.
- The live public asset versions after the restore are:
- styles.css?v=129 - app.js?v=137
- Public /api/status was revalidated after the restore and returned healthy V2.1 dataset metadata.
- English work was explicitly not committed and not deployed.
- Ongoing English localization now continues only in an isolated sibling workspace:
- llm_investigation_orchestrator_serbia_poc_english_wip_20260806_0718

Current English recommendation:

- continue UI/runtime localization first;
- do not begin scenario DB translation yet;
- only evaluate DB translation if English-mode users must read raw record text in English, or retrieval/export quality requires it.

What the isolated English WIP already covers:

- locale-toggle architecture with Hebrew as the default;

- additional runtime localization in app.js for shell/workstream/research/map/timeline labels;
- help.html locale-aware behavior through ?lang=he|en;
- continued reduction of static Hebrew dependence in the shell.

What still remains before an English deployment should be considered:

- more static-shell cleanup in index.html;
- final backend/agent locale-hardening across all reply paths;
- end-to-end English validation for switching, refresh, playback, workstreams, and agent messages.

Deployment packaging correction — 2026-08-06

An important deployment gap was discovered and corrected during the Hebrew restore.

The VM package must include not only the visible web assets, but also the runtime modules and scenario runtime content required by server.py.

At minimum include:

- server.py
- agent_routing.py
- agent_result_pipeline.py
- scenario_playback.py
- workstream_artifacts.py
- index.html
- app.js
- styles.css
- help.html
- README.md
- assets/
- vendor/
- data/
- recorded_runs/
- saved_questions/
- scenario_manifests/
- scenario_runs/

Operational notes:

- Preserve the existing VM /opt/serbia-poc-ui/.hermes-api.json; do not overwrite it with local placeholders.
- A partial package that omitted scenario_playback.py caused the UI service to fail with ModuleNotFoundError: No module named 'scenario_playback'.
- A partial package that omitted agent_routing.py would also silently miss current Moshe-routing behavior such as bilingual @משה / @Moshe mention handling.

Latest Update: REC-V2-007215 Final Validation

Playback foundation follow-up — 2026-07-28

Checkpoint 014 adds an inactive generic playback foundation:

- strict scenario manifests contain only scope and ordered timeframe stages;
- persistent runs support start/reopen, read, advance, complete, and reset;
- revisions and action-bound idempotency protect transitions;
- future stage windows are not exposed in API responses;
- all 93 automated tests pass.

Retrieval filtering, UI controls, automatic Moshe reevaluation, and deployment remain unimplemented. The Product owner requested confirmation before every implementation slice, so checkpoint 014 must be approved before Slice 2 begins.

Date: 2026-07-28

The deployed Moshe-to-workstream Phase 1 flow passed final validation using REC-V2-007215.

- An isolated workstream was created for the historical record.
- Moshe produced a non-persisting proposal with three REC indications,

uncertainty, contradictions, gaps, and human assessment questions.

- A distinct explicit confirmation created exactly one revision-1

target_assessment_lead artifact.

- The artifact survived a UI-service restart and reopen.
- The target-bank SHA-256 remained unchanged throughout.
- The validation workstream was archived after verification and reopens with

its artifact intact.

- All 85 automated tests pass.
- The release recommendation and operational latency note are recorded in

.ai/work/capabilities/shared-evidence-assessment/checkpoint-013.md.

Previous Update: Moshe Agent And Integrated Main

Date: 2026-07-21

PR #22 merged the latest Serbia POC capability line into main. The integrated state includes:

- the compact מכלול roster, reusable @member autocomplete, and header member conversation selection;
- real specialized routing for exact current-message @משה prompts, with isolated Moshe configuration and mission continuity;
- the V2.1 14,800-record dataset with evaluator-grounded fusion cases kept outside runtime data;
- constrained target fusion tools, a protected SQLite target bank, and a refreshable attack_targets catalog layer;
- shared General/Moshe answer and typed-layer presentation, including raw record references and source provenance;

- the existing investigation selector, explicit memory saving/restoration, step continuation, prompt-layer selection, and additive visualization model.

Production verification is technically complete. The recorded validation includes 53 focused Linux tests, JavaScript syntax and Python compilation, the complete 400-case evaluation, routing/session checks, read-only preparation, service health, permissions, isolated restore, evaluator isolation, and SQLite integrity. Final Product/QA capability acceptance remains pending.

Important interaction boundary:

- Moshe is a real specialized agent runtime.
- Selecting Talia, Naama, Gadi, or Yahli currently changes conversation presentation and attribution but still uses the General execution path; dedicated runtimes for those members do not yet exist.
- Team records remain predefined client-side members rather than authenticated users or a general agent identity model.

Authoritative capability artifacts:

- `.ai/work/capabilities/moshe-attack-targets/status.md`
- `.ai/work/capabilities/moshe-attack-targets/handoff-summary.md`
- `.ai/work/capabilities/moshe-attack-targets/checkpoint-009-slice-7-release.md`
- `.ai/work/capabilities/michlol-team-management/status.md`

Latest Update: Investigation Memory Slices 3-5 Review Deploy

Date: 2026-07-16

Current deployed asset versions after this update: `styles.css?v=81`, `app.js?v=103`, served from `/opt/serbia-poc-ui` on port 8769.

Current investigation memory behavior:

- Server-side investigation memory exists under `/opt/serbia-poc-ui/investigations/`.
- Final answers include an explicit `לזיכרון שמור` action.
- Clicking `לזיכרון שמור` appends a compact `chat_result_summary` through POST `/api/investigation-memory/chat-summary`.
- Layer tabs include an explicit `save-to-memory` action that appends `layer_filter_state` through POST `/api/investigation-memory/layer`.
- Saved layer memory includes label, kind, catalog layer id when available, source metadata, original/filtered counts, applied filters, and sample ids.
- Selecting an existing investigation loads server-side memory from GET `/api/investigation-memory?id=...`
- Catalog-backed saved layers reopen automatically through `/api/layers/<id>/rows` and restore their saved filters as applied filters.
- Saved result-derived layers without a catalog layer id remain context-only memory and are not reopened as UI tabs.
- Saved chat summaries and saved layer/filter memory are included in `investigation_state.saved_memory` for normal and continuation agent prompts.
- Saved-question behavior remains separate from investigation memory.

Review branch and PR:

- Branch: `codex/investigation-memory-slice2`
- Draft PR: <https://github.com/zvihev10/AI-Intelligence/pull/19>

VM verification performed after deployment:

- Public HTTP status returned 200 OK from `http://151.145.93.180/api/status`.
- Public index served `styles.css?v=81` and `app.js?v=103`.
- Public memory API smoke confirmed chat-summary save, layer save, and memory reload.
- Synthetic smoke investigation `investigation-vm-slice45-smoke` was removed from `/opt/serbia-poc-ui/investigations/`.

Previous Update: Investigation Selector First Slice

Date: 2026-07-16

Current deployed asset versions after this update: `styles.css?v=78`, `app.js?v=100`, served from `/opt/serbia-poc-ui` on port 8769.

Current investigation selector behavior:

- The active-investigation control moved to the upper header center.
- The header shows `פעילה חקירה` inline beside a simple investigation combo box.
- The analyst can type an investigation name; matching existing investigation names appear in a dropdown.
- Choosing an existing name switches the active investigation.
- Pressing `+` creates/selects the typed name as a new investigation; no additional modal or input window is used.
- The `+` tooltip/ARIA label is `חדשה חקירה צור`.
- Switching or creating an investigation clears the current workspace so layers/results from different investigations are not mixed.
- The previous conversation-panel title/subtitle area was removed, including the text `מעל טבעית בשפה חקירה` `הראיה למקורות ישיר מעבר עם, גולמיים אירועים`.

Implementation notes:

- Investigation registry is currently browser-local metadata in `localStorage` (`serbia-poc-investigations-v1`).
- This first slice does not yet persist separate per-investigation chat history, result layers, saved questions, recorded runs, or server-side state.
- The dropdown overlap bug was fixed by scoping combo-box button styling to `.investigation-combobox > button`, so dropdown option buttons are not styled as the main add button.
- `resetInvestigation({ keepInvestigation: true })` is used when selecting/creating an investigation to preserve the active investigation identity while clearing the visible workspace.

Future investigation-management work:

- Persist per-investigation chat/results/workspace state.
- Associate saved questions and recorded runs with `investigation_id`.
- Add backend/session support for investigation lists if cross-browser or multi-user persistence becomes required.
- Keep result layers as the source of truth for map/timeline/table filtering inside each active investigation.

Previous Update: Prompt Composer + Results Table Follow-up

Date: 2026-07-10

Asset versions from that update were styles.css?v=75, app.js?v=98, served from /opt/serbia-poc-ui on port 8769. Newer handoff sections supersede these versions.

Current prompt composer behavior:

- The composer is Codex-style: textarea above, plus button and up-arrow send button below.
- The plus button opens an options menu:
- **הקלטות** opens saved/replay investigations. - **שכבות** opens the floating layer-selection window.
- Prompt-layer selection is explicit and separate from map/table visibility.
- Visible/open layers are not attached to the agent prompt unless the user selects them through + -> **שכבות** and submits the layer-selection window.
- The selected-layer pill is hidden by default.
- After explicit selection, the pill appears under the text with selected-layer summary and a small ×.
- Clicking × clears prompt-layer selection and hides the pill, while keeping visible result/map/table layers open.

Current results-table behavior:

- Raw result tabs stay in one horizontal scroll row instead of wrapping/clipping in the fixed-height header.
- The raw results table exposes horizontal scrolling across viewport sizes.

Follow-up Fixes (2026-07-10)

- Closed GitHub issue #3 as completed for Multi-Layer Query Filtering.
- Changed the "מכאן המשך" step button to share the exact same pill styling as the neighboring **הצג / תוצאות** **הצג** / **תוצאות** **הצג** / **תוצאות** buttons.
- Strengthened step continuation so the original classify_question_intent summary, including recommended_mode and tool_budget, is copied into the continuation prompt and server-side continuation instructions.
- Fixed the **תוצאות** regression after step-result presentation by preserving per-step source IDs across continuation runs while still reconciling live-step IDs to the final run ID.
- Fixed the whole-query final-answer **הצג** **תוצאות** control so it switches to **תוצאות** after presenting final result layers and can hide/show those layers.
- Changed the prompt-area + button into an options menu with **הקלטות** for saved/replay runs and **שכבות** for choosing currently open table-capable layers in the same floating checkbox-window UX as step ingestion.

Previous Update: Step Injection ("מכאן המשך") Feature + Bug Fixes

Date: 2026-07-09

Bug Fixes (this session)

Step button label bug (was: always showing "תוצאות" after final answer returned)

Root cause: live-poll layers were keyed with state.investigationId; final result used result.run_id. After applyHermesResult rebuilt step DOM, resolvedStepSourceId() couldn't find layers because keys diverged.

Fix: applyHermesResult now rekeys both layer.sourceId AND layer.id at the moment the final result arrives (before renderActivitySteps). Both fields must be rekeyed together because addResultLayers deduplicates by item.id.

Duplicate layers bug (was: clicking step button after final answer added duplicate layers)

Same root cause as above. Fixed by the same rekey loop.

Filter Panel Simplification

- Removed "מסננים טיפוסים" title and "פעילים מסננים" section
- "מסנן הוסף" moved inline alongside "החל" button
- Empty placeholder text removed
- Empty middle section hidden when no draft filters exist
- Eye icon: white when layer visible, grey when hidden
- Filter icon: grey by default, white when any filters are applied (has-filters CSS class on the tab)

Step Injection Feature ("מכאן המשך")

Each step card now has a "מכאן המשך" button. Clicking it opens a floating window (right side, vertically centered) where the analyst can:

1. Write a free-text continuation instruction
2. Optionally select one or more visible table-capable layers to base the continuation on (up to 100 event IDs per layer are included in the prompt)

UI behavior:

- A new chat bubble is created for the continuation, labeled with a "חקירה המשך" kicker (CSS ::before on [data-continuation="true"])
- The bubble renders steps 1...N (where N = the step whose button was clicked) first, then appends new live steps N+1...M as they arrive
- result.investigation_steps is merged (prior + new) before applyHermesResult, so state.lastResult always holds the full chain

Server behavior (server.py):

- investigate() now accepts is_continuation=False parameter
- When is_continuation=True, the classify_question_intent instruction line is replaced (not just prepended) with a continuation directive:
- Tells Hermes not to call classify_question_intent - Tells Hermes to read the original classification from conversation history and preserve the established recommended_mode and tool_budget - Tells Hermes to continue directly from where the prior investigation ended

Client sends:

```
{
  "prompt": "...",
  "history": state.history,
  "investigation_id": state.investigationId,
  "is_continuation": true
}
```

Known behavior: The syntheticHistory approach (fake assistant ack turn) was removed. History is now sent as-is from state.history which already contains all prior user/assistant turns.

Step slice fix: openStepInjectModal stores stepNumber in stepInjectModal.dataset.fromStep. submitStepInject slices allPriorSteps.slice(0, fromStep) so the continuation bubble only shows prior steps up to and including the triggering step, not all original steps.

Semantic Tool Integration Quality

Date: 2026-07-06

Active branch:

feature/semantic-quality-tests

Latest relevant commits before this handoff update:

```
2e9f7ba Add configurable semantic embedding backend
a6d4eea Enable hybrid semantic retrieval by default
aa82add Make hybrid semantic cache engine-aware
7ea65ef Use hybrid semantic candidates in investigation tools
4747eb2 Add semantic tool integration comparison
```

Semantic retrieval status:

- semantic_search_events uses the shared SemanticEventIndex.
- Default backend is hybrid_embedding, which combines lexical TF-IDF recall with local dense/concept embedding reranking.
- lexical_tfidf remains available as a deterministic baseline/fallback through INTELLIGENCE_POC_SEMANTIC_BACKEND.
- Higher-level tool integrations now use the same backend:
- resolve_event_reference uses hybrid candidates to resolve vague analyst references to anchor events.
- trace_semantic_clues uses hybrid candidates over input clues, expanded clues, and seed summaries.
- find_related_events uses hybrid semantic similarity as a supporting "semantic" dimension, while structured bridges still dominate.

Quality artifacts:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/semantic_tool_integration_gold_v2.json
llm_investigation_orchestrator_serbia_poc/docs/quality/score_semantic_tool_integration.py
llm_investigation_orchestrator_serbia_poc/docs/quality/semantic_tool_integration_runs/
semantic_tool_integration_comparison_20260706T120341Z.md
llm_investigation_orchestrator_serbia_poc/docs/quality/semantic_tool_integration_runs/
semantic_tool_integration_comparison_20260706T120341Z.json
```

Comparison summary from the latest run:

- tp_13_resolve_shooting_reference / resolve_event_reference: previous baseline found 0/2 must-find records; current found 2/2; status PASS.
- tp_14_trace_tactical_noise_clues / trace_semantic_clues: previous baseline found 0/16; current found 16/16; status PASS.
- tp_15_related_from_zvecan_shooting_seeds / find_related_events: previous baseline already found 6/6; current still found 6/6 and now adds semantic bridge metadata; status PASS.

Rerun the semantic tool comparison from the Serbia POC directory:

```
cd llm_investigation_orchestrator_serbia_poc
$env:PYTHONIOENCODING='utf-8'
python docs/quality/score_semantic_tool_integration.py --current-label working_tree
```

Important sync lesson (2026-06-29): A stale local workspace and stale VM deployment briefly reintroduced old behavior: rectangular map markers and automatic final-answer presentation. GitHub already had the correct point-marker/manual-show behavior, but the VM was still serving older styles.css?v=36 and app.js?v=48. Before

every deploy, fetch GitHub, verify git status --short --branch, and deploy from the current committed main, not from stale uncommitted local files.

Latest UI completion note (2026-07-01): Map markers are colored points with popups; final-answer results are not presented automatically and are shown only via the final תוצאות הצג button. Result layers are shown in a flush transparent tabbed overlay attached to the map/timeline borders. Each layer appears as a real tab with a standard × close control, and the whole overlay uses standard window controls: – minimize, □ restore/maximize, and × close/clear. Step presentation controls moved into each step card: a text button toggles between תוצאות הצג and תוצאות הסתר, and שאילתה הצג opens the query modal. Modal close buttons now use the standard × icon instead of text סגור. Current deployed asset versions after the saved-question/close-icon work are styles.css?v=57 and app.js?v=77.

Active POC

Serbia / North Kosovo POC:

- Directory: llm_investigation_orchestrator_serbia_poc
- Local UI: http://127.0.0.1:8769/
- VM UI: https://151.145.93.180/
- Purpose: scenario-portability demo over North Kosovo escalation data, with analyst questions, data-grounded MCP tools, agent-step visibility, and map/timeline/table result presentation.

Cargo POC still exists but is not the active focus:

- Directory: llm_investigation_orchestrator_poc
- Local UI: http://127.0.0.1:8768/

Location And Entity Layer Normalization

Latest schema change, 2026-06-30:

The Serbia POC now treats entities exactly like locations.

Runtime event records contain stable foreign keys only:

```
event_id,timestamp_utc,source_type,source_reliability,source_reliability_label,certainty_level,entity_id,location_id,event_summary
```

Reference layers:

```
data/serbia_kosovo_locations.json
data/serbia_kosovo_entities.json
```

Important implications:

- entity_or_actor was removed from serbia_kosovo_events_projection.csv.
- ENTITY_REGISTRY was removed from mcp_server/server.py.
- data/serbia_kosovo_entities.json contains 16 entity records, one for each former raw actor value.
- Runtime event objects are enriched by the MCP loader with entity_name from the entities DB, the same way events are enriched with location_name from the locations DB.
- The active event object fields exposed to the agent/UI are now entity_id and entity_name, not entity_or_actor.
- get_events was removed. Use get_objects for all event/location/entity object retrieval.

- `get_objects(object_type="all", event_ids=[...])` returns the raw event objects plus their related location_layers and entity_layers.
- `aggregate_events(group_by="entity")`, `search_events(entity_ids=[...])`, `find_actor_history(entity_ids=[...])`, `find_related_events`, and `explain_linkage` all operate through entity IDs/names.
- actors remains only as compatibility input in some tool schemas; new prompts/tool calls should prefer `entity_ids`.

UI behavior:

- A single `הצג` action can add event, location, and entity layers together.
- Each layer is separate in the result tabs and can be hidden/closed independently.
- Entity layers render in the table and on the map through each entity's top locations.

Current VM Deployment

VM:

- Host: 151.145.93.180
- User: ubuntu
- SSH key path used locally: C:\Users\user\Downloads\oracle.key
- Important: user explicitly said not to touch/modify the key file.

Active UI service:

- Service: `serbia-poc-ui.service`
- Actual served path: `/opt/serbia-poc-ui`
- This is important: an earlier deploy mistakenly copied to `/opt/serbia-poc/ui`, but the active service serves `/opt/serbia-poc-ui`.
- Current source and deployed UI asset versions as of the Moshe release:
- `styles.css?v=88` - `app.js?v=109`
- These versions include the earlier additive-layer investigation experience plus the `מכלול` roster, member mention autocomplete, member selection, live Moshe opening/attribution, and shared attack-target presentation.

Active MCP/Hermes service:

- MCP path: `/opt/serbia-poc/mcp_server/server.py`
- Data path: `/opt/serbia-poc/data/serbia_kosovo_events_projection.csv`
- Active dataset: V2.1, 14,800 rows.
- General Hermes gateway service: `hermes-gateway.service`, local API port 127.0.0.1:8642.
- Moshe Hermes gateway service: `hermes-moshe-gateway.service`, local API port 127.0.0.1:8643.

Useful VM checks:

```
sudo systemctl is-active serbia-poc-ui.service
sudo systemctl is-active hermes-gateway.service
sudo systemctl is-active hermes-moshe-gateway.service
curl -k -fsS https://151.145.93.180/ | grep -E 'styles.css?v=|app.js?v='
curl -fsS http://127.0.0.1:8769/api/status
curl -fsS http://127.0.0.1:8769/api/live-steps
grep -n 'rawEventsOverlay|final-answer-show-btn|buildLocationLayer' /opt/serbia-poc-ui/app.js /opt/serbia-poc-ui/index.html
```

Deployment Notes

Use /opt/serbia-poc-ui for UI deploys.

When generating /opt/serbia-poc-ui/.hermes-api.json, write it as UTF-8 without BOM. The gateway now reads it with utf-8-sig, but no-BOM is still preferred.

Known fixed issue:

- A PowerShell-generated .hermes-api.json caused: Unexpected UTF-8 BOM (decode using utf-8-sig).
- Fix implemented in server.py: load_hermes_config() reads config using encoding="utf-8-sig".
- The deployed config was also rewritten without BOM.

Recommended UI deployment pattern:

1. Run git fetch origin main and confirm local main is aligned with origin/main.
2. Preserve the existing API key from /opt/serbia-poc-ui/.hermes-api.json or Hermes config.
3. Package committed files only: server.py, index.html, app.js, styles.css, help.html, README.md, vendor/, data/, recorded_runs/, and mcp_server/ when needed.
4. Copy to /opt/serbia-poc-ui.
5. Restart serbia-poc-ui.service.
6. Verify served versions through the public HTTPS endpoint, not only disk files.
7. Never deploy from a stale local working tree with uncommitted old UI files.

Saved Questions

Saved Questions are now the user-facing way to persist investigation results. After a successful live /api/investigate response, the final assistant answer shows תוצאות הצג and שמור side by side with the same pill-button look and feel. Saving writes the full result artifact, not only the final answer. The prompt input bar no longer contains a save button.

Runtime storage:

```
llm_investigation_orchestrator_serbia_poc/saved_questions/  
/opt/serbia-poc-ui/saved_questions/
```

Backend endpoints:

```
GET /api/saved-questions  
GET /api/saved-question?id=<saved-id>  
POST /api/saved-question  
DELETE /api/saved-question?id=<saved-id>
```

Implementation details:

- One UTF-8 JSON file per saved question.
- IDs are generated server-side as saved_YYYYMMDD_HHMMSS_<hex>.
- IDs are strictly validated before read/delete.
- Writes use a temporary file followed by rename.
- Listing skips corrupt or incomplete JSON files.
- Runtime saved_questions/*.json files are ignored by git; only .gitkeep is committed.
- Loading a saved question does not call Hermes. It restores the saved result through the normal applyHermesResult path so final-answer תוצאות הצג, per-step תוצאות הצג, map, timeline, table, event layers,

location layers, entity layers, and aggregation layers keep working.

- The final-answer שמור button changes שמור → שומר... → נשמר; failures show נכשל and restore the button after a short delay.

Deployment note: include saved_questions/ in the UI deployment package and ensure /opt/serbia-poc-ui/saved_questions/ is owned by the UI service user.

Recorded Demo Runs

Recorded demo runs are served by the UI backend from:

```
llm_investigation_orchestrator_serbia_poc/recorded_runs/
```

Each recording is a JSON file with this shape:

```
{
  "id": "q2_movement",
  "question": "analyst question shown in the replay modal",
  "title": "short title shown in the replay modal",
  "recorded_at_utc": "2026-06-28T12:56:18Z",
  "elapsed_ms": 171554,
  "source": "live_hermes_run_main_rerun_YYYYMMDD",
  "result": {
    "run_id": "run_...",
    "answer": "...",
    "event_ids": [],
    "answer_event_ids": [],
    "recommended_view": "map",
    "view_reason": "...",
    "investigation_steps": [],
    "events": [],
    "usage": {}
  }
}
```

The UI exposes recordings through:

```
GET /api/recorded-questions
GET /api/recorded-run?id=<recording-id>
```

The browser replays the saved investigation_steps at replay_delay_ms from /api/recorded-questions currently 2000 ms per step, then renders the saved final answer and result layers. Follow-up questions after a recording are sent as real live questions; the replayed user/assistant messages are added to chat history before the next live request.

Create or refresh a recording from a real local run:

```
cd llm_investigation_orchestrator_serbia_poc
$env:PYTHONPATH=(Resolve-Path ..\tools\python).Path
$env:PYTHONIOENCODING='utf-8'
& "C:\Users\user\cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" server.py
8769
```

In another PowerShell window:

```
$question = "בחרמבו נמזב תוחכונ תרבגה וא תוחוכ תעונת לש סופד תוהזל ותינ סאה?"
$body = @{
  prompt = $question
  history = @()
  investigation_id = "recording-q2-$(Get-Date -Format yyyyMMddHHmmss)"
} | ConvertTo-Json -Depth 20
$started = Get-Date
$result = Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:8769/api/investigate" -ContentType
"application/json; charset=utf-8" -Body $body
$elapsed = [int]((Get-Date) - $started).TotalMilliseconds
$recording = [ordered]@{
  id = "q2_movement"
  question = $question
  title = "תוחכונ תרבגהו תוחוכ תעונת"
  recorded_at_utc = (Get-Date).ToUniversalTime().ToString("o")
  elapsed_ms = $elapsed
```

```

source = "live_hermes_run_main_rerun_$(Get-Date -Format yyyyMMdd)"
result = $result
}
$recording | ConvertTo-Json -Depth 100 | Set-Content -Encoding utf8 "recorded_runs\q2_movement.json"

```

After saving a recording, verify it locally:

```

Invoke-RestMethod "http://127.0.0.1:8769/api/recorded-questions"
Invoke-RestMethod "http://127.0.0.1:8769/api/recorded-run?id=q2_movement"

```

Then open <http://127.0.0.1:8769/>, press the + recorded-questions button, and replay the refreshed question. Check that:

- The question appears in the modal.
- Steps replay at roughly two seconds per step.
- Final answer, event IDs, map/timeline/table layers, and per-step הציג all work.
- The saved result does not contain failed/partial runs.

Deploy recordings with the UI. Include the whole recorded_runs/ directory in the package copied to /opt/serbia-poc-ui/recorded_runs. No Hermes/MCP restart is required for recording-only changes; restart serbia-poc-ui.service because recordings are served by the UI backend.

Recommended VM recording verification:

```

curl -k -fsS https://151.145.93.180/api/recorded-questions
curl -k -fsS "https://151.145.93.180/api/recorded-run?id=q2_movement" | head -c 500

```

Do not generate demo recordings from synthetic fallback data or manually shortened answers. The recording must come from a real /api/investigate Hermes run so it preserves the actual answer, step explanations, tool outputs, result layers, and usage/performance metadata.

Current UI Architecture

The UI has been refactored from “views over current records” into an additive layer-based result model.

Core idea:

```

tool/final result
-> result layers
-> map/timeline/table render each visible layer according to capability

```

Layer identity model:

- Every displayed layer has a sourceId and a dataId.
- sourceId identifies where the layer came from: a specific final assistant answer or a specific investigation step.
- dataId identifies the visual data inside that source, for example event-source layer, location-summary layer, date aggregation, or generic aggregation.
- The concrete layer key is built from sourceId + dataId.
- Pressing הציג is additive: it adds related layers without replacing currently displayed layers.
- Pressing the same הציג again re-shows/focuses existing layers instead of duplicating them.
- If a layer was closed with x, pressing the same הציג recreates it.
- Layer colors are assigned automatically from a palette and released when the layer is closed.

Current layer types:

- events
- One layer per source_type, e.g. טלגרם, X, מקומיות חדשות. - Capabilities: table, map, timeline.

- locations
- Location-summary layer, e.g. מיקומים ריכוזי. - Capabilities: table, map. - Not filter-derived from source tabs unless backing events exist in an event layer.
- time_aggregation
- Time aggregation layer, e.g. summary by date/hour. - Capabilities: table, timeline.
- group_aggregation
- Generic aggregation layer, e.g. grouping by actor/source/category when it is not a map or timeline group. - Capabilities: table.

Important functions in app.js:

- buildEventLayers(events)
- buildLocationLayer(locations)
- buildTimeAggregationLayer(items)
- buildGroupAggregationLayer(items)
- buildResultLayers(...)
- addResultLayers(...)
- finalSourceId(result)
- stepSourceId(resultOrBase, stepNumber)
- visibleLayers(capability)
- activeTableLayer()
- renderMap()
- renderTimeline()
- renderEvidence()

Current UI behavior:

- The raw/results table is a transparent overlay shared by map and timeline, flush with the map/timeline borders.
- Table tabs are real layer tabs, not source-type-only pills.
- Each layer tab has a standard eye/eye-off toggle.
- Each layer tab has a standard × close control.
- Each layer tab displays the layer color.
- Hiding a layer affects all visualizations where that layer participates.
- Closing a layer removes it from the current workspace and releases its color.
- The overlay can be resized, minimized with –, restored/maximized with □, and closed/cleared with ×.
- Final assistant answers have a תוצאות הצג button styled like the step action pills.
- Clicking final תוצאות הצג adds or focuses that answer's result layers without automatically overriding the current visual state on answer arrival.
- Tool steps use תוצאות הצג / תוצאות הסתר for step result layers and שאילתה הצג for query/tool details.
- Map locations are shown as colored point markers, not always-open rectangles.
- Clicking a map point opens a MapLibre popup with the location name, item count, and contributing layer labels.

Removed UI sections:

- Removed the former result summary header (מצב תמונת, ממצאי, הסוכן חקירת ממצאי, event count badge) from the top of the result panel.
- Removed the old separate גולמיים אירועים top-level view tab.
- Removed extra bottom query/layer labels that duplicated information.

Data Normalization

The active data was normalized to remove confusing/non-real source categories.

Removed field:

- source_category

Removed source/source-like labels from active data:

- מטעה מידע/דיסאינפורמציה
- קשור לא רעש
- אזרחי דיווח
- חירום דיווח

Approved visible source_type values:

- פייסבוק
- מקומיות חדשות
- X
- פוליטי בלוג
- טלגרם
- דובר הודעת
- טיקטוק
- בינלאומי חדשות ערוץ
- מקומית שמועה
- וואטסאפ קבוצת

Active source counts after normalization:

תיומקם תושדח 1307
פרגלט 1280
קוטקייט 1074
תיומקם העומש 1017
970 X
פאסטאוו תצובק 912
רבוד תעדזה 887
יטילופ גולב 875
קובסייפ 870
ימואלניב תושדח פורע 808

Normalized active files:

- data/north_kosovo_attachment_inspect/north_kosovo_synthetic_dataset_he_10k_subset.csv
- data/north_kosovo_attachment_inspect/north_kosovo_synthetic_dataset_he_10k_subset.jsonl
- data/serbia_kosovo_events_projection.csv

- data/serbia_kosovo_evaluator_labels.csv

Normalizer/report:

- data/normalize_sources.py
- data/source_normalization_report.json

Validation already performed:

- Active data row count preserved: 10000.
- Core fields preserved: timestamp, location_id, event_id, text.
- No active-data matches for removed labels or source_category.

Historical backup/test/recorded files may still contain old terms. Do not treat those as active runtime data.

Model Prompt And MCP Schema Updates

The model/gateway prompt and MCP schema were updated so source examples match the normalized data.

Updated:

- llm_investigation_orchestrator_serbia_poc/server.py
- llm_investigation_orchestrator_serbia_poc/mcp_server/server.py

Removed stale examples such as:

- SIGINT
- financial alerts
- sensors
- port/customs records

Current guidance uses visible channels such as:

- טלגרם
- טיקטוק
- X
- פייסבוק
- מקומיות חדשות
- דובר הודעת
- וואטסאפ קבוצת
- מקומית שמועה
- פוליטי בלוג
- בינלאומי חדשות ערוץ

MCP Tools

Current Serbia MCP tools:

```
classify_question_intent
plan_next_investigation_step
search_events
semantic_search_events
```

get_objects
resolve_location
resolve_event_reference
find_actor_history
aggregate_events
explain_linkage
build_event_sequence
resolve_entity
trace_identifier
trace_semantic_clues
find_related_events
compare_location_claims
challenge_hypothesis

Tool outputs can include:

- events / event_ids
- locations / location_ids
- location_layers
- entity_layers
- map_locations
- aggregate_groups
- route
- ordered_event_ids
- recommended_next_seeds
- new_clues_to_trace
- conflict_groups
- alternative_events
- gaps
- bridges
- entity resolver matches

Visualization architecture now treats displayable outputs as layers.

First-class display-layer candidates:

- events
- locations
- entities
- time aggregations

Potential future layer types:

- routes/sequences
- conflict groups
- link/bridge relationships
- semantic clue clusters

Entities

Entities are now first-class reference-layer objects, symmetric with locations.

Active data fields:

- Projection CSV has entity_id.
- Projection CSV no longer has entity_or_actor.
- data/serbia_kosovo_entities.json contains the 16 entity rows used by the projection.
- MCP enriches public events with entity_name from the entities DB.
- Tools should prefer entity_ids; actors is compatibility input only where still present.

Current entity reference file:

llm_investigation_orchestrator_serbia_poc/data/serbia_kosovo_entities.json

ENTITY_REGISTRY was removed from MCP code. Entity IDs are DB-backed, not hardcoded registry aliases.

Data Policy

Visible runtime data:

- serbia_kosovo_events_projection.csv
- serbia_kosovo_locations.json

Evaluation-only hidden labels:

- serbia_kosovo_evaluator_labels.csv

Do not expose evaluator labels to the orchestrator, MCP runtime, UI, or prompt. They include ground truth and misleading labels that would contaminate the intelligence-analysis scenario.

Current Result Presentation Behavior

Current behavior:

- Tool-step הצג presents the selected step's results/layers.
- Final-answer תוצאות הצג presents/restores the final answer's result layers.
- Final answer arrival does not automatically replace the current map/timeline/table presentation. This is intentional: the analyst decides when to present the final answer's layers.
- applyHermesResult(..., { restoreOnly: true }) is the path used by final תוצאות הצג to build/show final-answer layers.
- Normal final-answer arrival only finalizes the chat answer and keeps visualization state under user control.

Design decision:

- Preserve the current step/layer view until the user clicks final תוצאות הצג.
- Do not silently override an explicitly selected tool-step view.
- Keep final תוצאות הצג as the reliable manual restore/presentation action.

Phase 2: Query Builder — Editable Query Form

Status: Step 1 UI foundation and query edit modal controls exist in current main. Future work should continue from the committed GitHub state, not from older local snapshots.

Objective: Transform the query display from read-only JSON (<pre> modal) to an editable form component with smart "Run New Query" button visibility.

Key Changes in Step 1:

1. Remove result data from query object: event_ids, map_locations, aggregate_groups no longer appear in queryReadoutForLayer() payload.
2. Replace modal <pre> element with form-based modal containing:
 - Tool name (read-only display) - Layer selector (editable dropdown: map, timeline) - Arguments editor (editable textarea with JSON) - "חדשה שליפה תריץ" button (hidden until user edits)
3. Add state tracking: state.queryEdited boolean, change detection on form inputs.
4. Add form styling to match existing UI palette.
5. Stub handler handleQueryFormSubmit() for Phase 2a.

Files affected:

- app.js (query cleanup, query form state/functions, modal controls, step-card visibility behavior, layer presentation behavior; current deployed version v=68)
- index.html (query modal/result controls; current deployed script version v=68)
- styles.css (query form, layer tabs, point markers, result-window controls, standard eye/eye-off visibility icons and step-result text toggle styling; current deployed version v=52)

Rationale:

- Query ≠ Results: Query payload should contain only request parameters, not response data.
- Form-based UI: Editable fields are more intuitive than JSON text.
- Smart visibility: Run button appears only after edits, preventing accidental re-runs of unchanged queries.
- Foundation for Phase 2: Prepares UI for spatial query type selector, temporal range picker, and filter dropdowns.

Full plan: Continue from this handoff and the committed code in main; do not rely on older private scratch files from another Windows user profile.

Next phase after Step 1:

- Phase 2a: Implement handleQueryFormSubmit() to call agent with edited query and create new layer.
- Phase 2b: Add spatial query type selector + draw tools (proximity, polygon, corridor).
- Phase 2c: Add temporal range picker.
- Phase 2d: Add filter dropdowns (source, certainty, labels).

Near-Term RAG / Real Semantic Search Plan

The team agreed to implement a small local RAG capability before changing more investigation behavior. The goal is not Elastic/OpenSearch yet; it is to replace hardcoded semantic-style retrieval with one shared semantic index over real event data.

This plan must use the normalized entity/location schema from this branch:

- Runtime events contain entity_id and location_id.
- Entity names and aliases come from data/serbia_kosovo_entities.json.
- Location names and coordinates come from data/serbia_kosovo_locations.json.

- Runtime events expose `entity_name` and `location_name` only after MCP enrichment.
- `get_events` no longer exists; use `get_objects`.

Original problem:

- Several tools used the word “semantic”, but were mostly hardcoded keyword/clue matching.
- `trace_semantic_clues` retrieved by user/seed clues plus `SEMANTIC_CLUE_TERMS`.
- `find_related_events` had a semantic dimension, but it meant shared hardcoded clue terms.
- `explain_linkage` used hardcoded clue overlap for `semantic_overlap`.
- `compare_location_claims` used seed-derived clue keywords and `GEO_CONFLICT_MARKERS` for retrieval/grouping.
- This was useful and explainable, but it was not real semantic similarity.

Implementation status:

- Phase 1 foundation is implemented as `mcp_server/semantic_index.py`.
- The default backend is now `hybrid_embedding`: lexical TF-IDF candidate recall plus local dense/concept embedding reranking.
- `lexical_tfidf` remains available as a baseline/fallback.
- Public MCP tool `semantic_search_events` is implemented and returns normal public event objects plus `semantic_score` and a short rationale.
- The orchestrator prompt tells the model to use `semantic_search_events` for fuzzy/paraphrased retrieval, and not for exact IDs, exact filters, aggregation, or object retrieval.
- `resolve_event_reference`, `trace_semantic_clues`, and `find_related_events` now use the shared semantic backend internally.
- `explain_linkage` and `compare_location_claims` still use mostly deterministic/marker logic and are candidates for later semantic integration after a separate quality baseline.

Target architecture:

1. Add one shared internal semantic backend: `SemanticEventIndex`.

- Build/load semantic retrieval vectors for all active event records. - Current implementation uses lexical TF-IDF; future implementation can swap in multilingual embeddings/vector similarity. - Apply metadata filters before/after retrieval: `time`, `location_id`, `entity_id`, `source_type`, `reliability/certainty`. - Return scored event candidates with canonical REC-* IDs. - Store no anonymous chunks; every result must map back to an event row.

2. Add a public MCP tool: `semantic_search_events`.

- Input: natural-language query, optional `seed_event_ids`, filters, and limit. - Filters should align with current tools: `start_time`, `end_time`, `location_ids`, `entity_ids`, `source_types`, `reliabilities`, `keywords`, and `limit`. - Output: `event_ids`, events, semantic scores, and concise match rationale. - UI should render returned events as normal result layers/tabs/map/timeline records. - This tool is the visible interface; existing tools should call the same internal `SemanticEventIndex` directly, not call the MCP tool as an external client.

3. Keep deterministic search tools.

- Do not replace `search_events`, `aggregate_events`, `get_objects`, `resolve_location`, `resolve_entity`, or `trace_identifier`. - Exact filters, IDs, aliases, location/time constraints, and aggregation remain deterministic. - RAG replaces fake semantic retrieval only, not exact search.

Implementation phases:

Phase 1 — Semantic index foundation

- Create an event text builder using enriched public event fields:
- event_summary - entity_id - entity_name - location_id - location_name - source_type - timestamp_utc - certainty_level - source_reliability_label
- Do not use removed fields such as entity_or_actor.
- Current local POC stack: persisted sparse lexical TF-IDF index, no third-party dependency.
- Future stack option: small multilingual embedding model plus hnswlib, FAISS, or a simple persisted embedding matrix if deployment constraints allow it.
- Add index build/load path that can regenerate from CSV plus entities/locations JSON during deploy/startup.
- Keep metadata sidecar keyed by event_id.

Phase 2 — Public semantic tool

- Add semantic_search_events to mcp_server/server.py.
- Support filters compatible with search_events: time, location IDs, entity IDs, source types, reliability/certainty, and limit.
- Return normal public events plus semantic_score.
- Add audit output and tool schema.
- Add basic smoke/regression coverage.

Phase 3 — Convert fake-semantic tools

- trace_semantic_clues
- Done for retrieval: it now uses the shared semantic backend. - Keep negation/benign/direct-observation markers only as scoring/explanation modifiers. - Consider later rename to trace_operational_clues if the public API should be clearer.
- find_related_events
- Keep deterministic dimensions: identifier, entity, time, location. - Entity dimension must use entity_id and entity aliases from serbia_kosovo_entities.json. - Done: the semantic dimension now uses shared semantic similarity as a supporting bridge signal. - Hybrid score should combine deterministic bridges plus semantic score. - Do not let semantic similarity outrank exact identifier evidence by default.
- explain_linkage
- Keep exact bridges: identifier, entity_id, time, location. - Replace hardcoded semantic_overlap with event-vector similarity. - Return similarity score and a cautious rationale; do not claim causal truth.
- compare_location_claims
- Keep geographic-conflict grouping and GEO_CONFLICT_MARKERS as warning/explanation signals. - Replace seed-derived keyword retrieval with semantic retrieval of similar claim narratives across locations. - Continue to state that the tool has no ground truth and cannot decide the correct location.

Phase 4 — Cleanup and naming

- Remove or demote SEMANTIC_CLUE_TERMS from remaining retrieval paths after the semantic backend is stable.
- Keep BENIGN_MARKERS, NEGATION_MARKERS, DIRECT_OBSERVATION_MARKERS, and GEO_CONFLICT_MARKERS only as explainable scoring/warning signals.
- Update tool descriptions so no tool implies hardcoded clue matching is true semantic search.
- Update orchestrator guidance to call semantic_search_events when wording mismatch/fuzzy recall is likely.

Design constraints:

- RAG finds evidence; it does not decide truth.
- The LLM must not receive raw vector DB access.
- Hidden scenario/evaluator labels must never be used for retrieval.
- All semantic results must remain auditable through REC-* IDs.
- UI behavior stays layer-based: semantic results become ordinary event result layers/tabs by source type or result grouping.
- RAG result events must preserve enriched entity_id, entity_name, location_id, and location_name.
- Elastic/OpenSearch remains out of scope until corpus size, concurrency, or ops needs justify it.

RAG Quality Test Catalog

Step 1 of the deeper quality-test plan is defined here:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/question_catalog_v1.json
```

It contains:

- 8 full investigation questions for the app/Hermes/agent path.
- 10 direct tool-level probes for MCP/tool isolation.

The companion explanation is:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/quality_test_plan.md
```

Step 2 reference base is now here:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/reference_base_v1.json
```

Current status:

- fq_01 through fq_05 have current baselines from existing real recordings.
- Enabled tool probes have direct MCP current-output baselines.
- Ideal targets define criteria and expected output behavior, but reviewed must_find / acceptable / must_not_prioritize ID lists still need an analyst/offline-label pass.
- Semantic integration probes tp_13, tp_14, and tp_15 were added to cover the tools changed in the hybrid semantic work.
- The semantic integration gold/reference file and scorer are:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/semantic_tool_integration_gold_v2.json  
llm_investigation_orchestrator_serbia_poc/docs/quality/score_semantic_tool_integration.py
```

Latest saved comparison:

```
llm_investigation_orchestrator_serbia_poc/docs/quality/semantic_tool_integration_runs/  
semantic_tool_integration_comparison_20260706T120341Z.md
```

Validation Commands

From repo root:

```
cd llm_investigation_orchestrator_serbia_poc  
node --check app.js  
& "C:\Users\user\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" -m  
py_compile server.py mcp_server/server.py mcp_server/benchmark_tools.py mcp_server/regression_quality.py  
git diff --check
```

MCP smoke test:

```
$env:PYTHONIOENCODING='utf-8'
& "C:\Users\user\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" mcp_server\
smoke_client.py
```

Data validation:

```
Select-String -Path `
  data\north_kosovo_attachment_inspect\north_kosovo_synthetic_dataset_he_10k_subset.csv,`
  data\north_kosovo_attachment_inspect\north_kosovo_synthetic_dataset_he_10k_subset.jsonl,`
  data\serbia_kosovo_events_projection.csv,`
  data\serbia_kosovo_evaluator_labels.csv `
-Pattern 'source_category|שער|העטמ עדימ|היצמרופניאסיד|מוריח חוויד|יחרזא חוויד|רושק אל' -SimpleMatch
```

Expected: no matches in active data files.

Known Risks / Watch Items

1. Layer architecture is new.
- Test map/timeline/table after each result path: live answer, recorded answer, tool-step הוצג, final-answer הוצג.
2. Static location layers are intentionally not source-filtered.
- They are peer layers, not derived event filters. - Eye toggle hides/shows the whole location layer.
3. VM deploy path and stale-code confusion.
- Use /opt/serbia-poc-ui, not /opt/serbia-poc/ui. - Verify public served asset versions after deploy. - If the UI shows rectangles instead of colored points, the VM is serving stale CSS/JS. - If final answers auto-present results, the VM/local code is stale; current main requires manual final הוצג.
4. Config BOM.
- Write .hermes-api.json without BOM. - server.py now tolerates BOM via utf-8-sig.
5. Historical files still contain old source labels.
- Active data is clean; historical test/recorded files may not be.
6. Entity IDs are now first-class runtime data.
- Event rows contain entity_id. - Entity names and aliases come from data/serbia_kosovo_entities.json. - Treat stale references to entity_or_actor as legacy-only unless inspecting archived data.

Suggested First Message To A New Assistant

Read PROJECT_HANDOFF.md first, then the status and handoff artifacts for the capability you are changing. Continue work on the Serbia/North Kosovo POC in llm_investigation_orchestrator_serbia_poc. The UI is deployed from /opt/serbia-poc-ui on VM 151.145.93.180 (port 8769) and the merged source references styles.css?v=88 and app.js?v=109. Do not modify any SSH key file.

Current behavior: colored map point markers with popups; final answers do not auto-present visualization layers; final תוצאות גצה presents/restores final-answer layers manually. The result table is a flush transparent tabbed overlay with real horizontally scrollable layer tabs, standard eye/eye-off toggles (white when active, grey when inactive), per-tab filter icon (white when filters applied), per-tab 'x' close, resize, '-' minimize, 'm' restore/maximize, window 'x' close/clear, and horizontal table scrolling. Filter panel is simplified: no title/active-section, inline לחץ/סיום buttons.

Prompt composer behavior: the prompt box is Codex-style with plus and up-arrow controls under the text. The plus menu has תוסלקה and תובכש. Prompt-layer context is attached only after the user explicitly selects layers through '+' -> תובכש; visible/map/table layers alone are not sent as selected prompt layers. The selected-layer pill is hidden until selection and includes a small 'x' to remove the prompt-layer option.

Each step card has a "ואנכ רשמה" button that opens a floating window for agent continuation. Continuation sends is_continuation:true to server.py which replaces the classify_question_intent instruction with a directive to continue from existing history using the original mode/budget. The

continuation bubble renders all prior steps (up to the triggering step) plus new live steps, merged into state.lastResult.

The UI uses an additive source/data layer architecture; preserve that model. V2.1 is active with 14,800 records. General uses port 8642 and exact current-message @השם routing uses the isolated Moshe runtime on port 8643. Moshe is a real specialized agent; the other selected members remain presentation-only over the General path. Before deploying, fetch GitHub and verify the VM is not serving stale assets. The authoritative integrated branch is main.

File Review Order

For UI/layer work:

Preserve the additive layer model:

- Keep visualization state decoupled from chat state.
- Use sourceId for final-answer/step origin and dataId for the layer data identity.
- הצג should add/focus related layers, not replace unrelated visible layers.
- Layer colors are workspace-level visual identities and should not be tied to chat order.

1. llm_investigation_orchestrator_serbia_poc/index.html
2. llm_investigation_orchestrator_serbia_poc/styles.css
3. llm_investigation_orchestrator_serbia_poc/app.js
4. llm_investigation_orchestrator_serbia_poc/server.py

For agent/tool behavior:

1. llm_investigation_orchestrator_serbia_poc/server.py
2. llm_investigation_orchestrator_serbia_poc/mcp_server/server.py
3. llm_investigation_orchestrator_serbia_poc/mcp_server/smoke_client.py
4. llm_investigation_orchestrator_serbia_poc/mcp_server/regression_quality.py

For data normalization:

1. llm_investigation_orchestrator_serbia_poc/data/normalize_sources.py
2. llm_investigation_orchestrator_serbia_poc/data/source_normalization_report.json
3. Active CSV/JSONL files under llm_investigation_orchestrator_serbia_poc/data/

Final Sanity Checklist Before Future Handoff

- Run git status --short --branch.
- Verify served VM asset versions through the public HTTPS endpoint, not just disk.
- Verify serbia-poc-ui.service is active.
- Verify /api/live-steps returns JSON.
- If MCP changed, run smoke test before deploy.
- If data changed, run active-data forbidden-label search.
- Do not expose evaluator labels.
- Do not touch the SSH key file.